

Towards Role-Based Multi-Agent LLM Systems for Software Requirements Analysis

Novica M. Trifković, Ilija D. Antović

Abstract— This paper presents a structured overview of recent research on the application of Large Language Models (LLMs) and multi-agent systems in software requirements analysis and introduces a conceptual, role-based multi-agent architecture for addressing identified limitations. The proposed approach emphasizes hierarchical supervision, collaboration between Large and Small Language Models, and structured cross-checking based on report comparison and uncertainty-aware arbitration to improve robustness and reliability. The contribution of this work is theoretical and exploratory, providing a foundation for future research on multi-agent LLM systems aimed at improving the software requirements analysis process.

Key words: software requirements analysis, LLM, SLM, Multi-agent

I. INTRODUCTION

Software requirements analysis represents one of the most critical and error-prone phases of the software development lifecycle. Decisions made at this stage directly influence system architecture, implementation effort, cost, and long-term maintainability, while errors introduced early often propagate to later development phases with significant negative consequences [1]. According to the Software Engineering Body of Knowledge (SWEBOK) [2], requirements analysis encompasses activities such as basic requirements analysis, addressing quality-of-service constraints, formal analysis, and conflict resolution, all of which require a high degree of precision, consistency, and domain understanding. In practice, however, requirements are predominantly expressed in natural language, which makes the analysis process inherently ambiguous, time-consuming, and susceptible to inconsistencies and errors.

Large Language Models (LLMs) have emerged as a promising technological enabler for supporting and partially automating software engineering activities, including requirements analysis [3]. Due to their ability to process and interpret natural language, LLMs have demonstrated significant potential in extracting, summarizing, and structuring information from textual requirements. Recent

surveys and literature reviews indicate that LLM-based systems are increasingly applied across various software engineering tasks, highlighting their capability to understand, reason about, and generate software-related artifacts [4][5]. Furthermore, the integration of LLMs into autonomous agents and multi-agent systems (MAS) has been identified as a significant shift in the research landscape, enabling collaborative reasoning, coordination, and task decomposition for complex problem solving [6].

LLM-based agents have found applications in different stages of complex problem solving. According to Li et al. [4], LLM-based MAS are considered a promising pathway towards realizing general artificial intelligence that is equivalent to or surpasses human-level intelligence. Such systems can perceive, reason, and act collaboratively, often outperforming single-agent approaches in complex domains [4][7]. Existing research identifies five key components of LLM-based MAS architecture: profile, perception, self-action, mutual interaction, and evolution, each describing how agents react and adapt within a given environment [4]. However, this interaction-driven behavior may lead to suboptimal decisions.

Despite their potential, MAS remains limited in solving certain classes of problems. To address scalability and coordination challenges, concepts such as *MegaAgent*, a large-scale autonomous LLM-based system capable of dynamically creating agents based on task complexity, have been proposed [8]. Further advances include decentralized coordination paradigms such as *AgentNet*, which introduce dynamic agent graph topologies and retrieval-based memory mechanisms [9]. Although recent progress has significantly enhanced the ability of multiple LLM-based agents to perceive, learn, reason, and act collaboratively [7], several challenges remain unresolved, including hallucination, limited learning from immediate feedback, and high computational resource requirements caused by model scaling [10]. These issues are particularly problematic in the context of software requirements analysis, where errors and ambiguities may propagate into later development phases with significant negative consequences.

To mitigate some of these limitations, recent research has explored the role of smaller, task-specialized language models, commonly referred to as Small Language Models (SLMs). Compared to LLMs, SLMs are often more efficient, context-focused, and better suited for narrowly defined

*Research supported by ABC Foundation.

Novica Trifković is a PhD student at the Faculty of Organizational Sciences, University of Belgrade (phone: +381/65-354-7340; e-mail: novicatrifkovic@gmail.com).

Ilija Antović is an Associate Professor at the Faculty of Organizational Sciences, University of Belgrade, Department of Software Engineering (e-mail: ilija.antovic@fon.bg.ac.rs).

analytical tasks [11]. Schick&Schütze [12] identified key factors required for successful natural language understanding using small language models. Despite their context focus, SLMs struggle with generalization and robustness.

Empirical studies in software requirements engineering confirm that, while language models can significantly improve efficiency, they do not fully eliminate issues related to clarity, consistency, and reliability. For example, existing work on automated user story generation and prioritization shows that LLM-based approaches can produce functionally relevant requirements with high semantic similarity, yet still suffer from variability across runs and inconsistencies in expression [13]. Similarly, frameworks such as *MetaGPT* attempt to address cascading hallucinations and logical inconsistencies by introducing standardized operating procedures and structured collaboration among agents [14]. Despite these advances, a comprehensive understanding of the effects, limits and reliability of LLM-based approaches in software engineering remains at an early stage [5].

These observations motivate a hybrid perspective in which LLMs and SLMs are combined within a coordinated multi-agent system, leveraging their complementary strengths. In such a system, LLMs operate at a higher level of abstraction, providing global understanding, supervision, and orchestration, while SLMs act as specialized agents responsible for well-defined subtasks. This naturally leads to a role-based multi-agent architecture, in which responsibilities are distributed according to the level of abstraction and analytical scope. However, introducing multiple agents also raises challenges related to coordination, validation, and decision reliability. Disagreements between agents, inconsistencies in generated artifacts, and uncertainty in analytical outcomes must be explicitly addressed. One possible approach is to introduce systematic cross-checking mechanisms, where the outputs of multiple specialized agents are compared and evaluated under the supervision of a higher-level agent. Such mechanisms can be interpreted as a form of role-based collaboration, where agents operating at different levels of abstraction mutually reinforce or challenge each other's conclusions to reduce ambiguity and error propagation.

The goal of this paper is to provide a structured overview of existing research on the application of large language models and multi-agent systems in software requirements analysis, and to propose a conceptual, role-based architecture as a research vision for improving the process of software requirements analysis. The contribution of this work is predominantly theoretical and exploratory. Rather than presenting a fully implemented framework or empirical evaluation, the paper identifies key limitations of current approaches and outlines a conceptual foundation for multi-agent, LLM-enabled systems supporting more reliable requirements analysis. Specifically, it proposes a role-based agentic architecture, defines a redundancy-based double-checking mechanism supervised by an LLM, introduces a

task-level decomposition of requirements analysis, and provides a probabilistic reliability formulation explaining how redundancy may reduce cumulative error.

II. MATERIALS AND METHODS

In this research the model of organizing the relationship between the LLM and SLMs is proposed to obtain acceptable results in the process of software requirement analysis. According to the SWEBOK guide established by IEEE Computer Society [2], it is possible to state that the software requirements analysis is almost the most important part in the process of software design. It encompasses four steps: Basic Requirements Analysis, Economics of Quality-of-Service Constraints, Formal Analysis, and Addressing Conflict Requirements. Within this decomposition, each step defines a distinct analytical role in the system, while concrete tasks represent instances of applying that role to particular requirement artifacts serving as input. The agents generate role-specific structured outputs, whose form and granularity depend on the analytical responsibility of the assigned role. To reduce individual model errors, tasks are executed in parallel by paired SLM agents, and their outputs are compared using a predefined similarity thresholds (ϵ). In cases of disagreement, a supervisory LLM performs arbitration through contextual reasoning and refines or selects the final result. Although this represents only the “top of the iceberg” in requirements engineering, the emphasis here is not on detailing all possible roles, but on proposing a redundancy-based validation approach to reduce errors and ambiguity. In mathematical interpretation it could be written as follows:

$$\delta = |R_1 - R_2| \quad (1)$$

$$D_1 = \begin{cases} \delta \leq \epsilon, & \text{the reports are accepted} \\ \delta > \epsilon, & \text{further analysis required} \end{cases} \quad (2)$$

where:

- δ – distance metric between two reports;
- R_1 – vector representation of SLM₁ output report;
- R_2 – vector representation of SLM₂ output report;
- D_1 – decision in the first phase and
- ϵ – predefined distance threshold.

In the proposed model, relations between the LLM (as the supervisory decision-maker) and SLM agents are organized hierarchically, with tasks divided according to their hierarchical level and analytical scope. The underlying principle can be formulated as follows: higher levels of abstraction enable broader contextual validation in decision making. Higher-level agents may challenge and validate the reports generated by lower-level agents in order to reinforce logical consistency, reduce ambiguity, and minimize analytical inaccuracies. In this model, σ represents an uncertainty metric derived from model confidence estimates, where smaller values of σ correspond to higher reliability. In mathematical interpretation this could be written as follows:

$$D_2 = \begin{cases} \sigma_{LLM} < \sigma_{SLM}, & \text{the reports are accepted} \\ \sigma_{LLM} \geq \sigma_{SLM}, & \text{further analysis required} \end{cases} \quad (3)$$

where:

- D_2 – supervisory decision performed by the LLM;

- σ_{LLM} – uncertainty of the LLM output, and
- σ_{SLM} – uncertainty of the SLM output.

In this model, the LLM operates as a supervisory agent responsible for evaluating the outputs generated by SLM agents. Upon receiving a report from an SLM, the LLM assesses its logical consistency, identifies potential ambiguities, and evaluates the risk of error propagation in subsequent analytical steps. Double checking therefore consists of two complementary levels of validation: report-level comparison based on the distance threshold defined in formulas (1) and (2), and supervisory arbitration based on the uncertainty comparison defined in formula (3). The supervisory decision is guided by the uncertainty measures associated with the LLM and SLM outputs. Since smaller values of σ correspond to higher reliability, the model favors the result associated with lower uncertainty. In this way, double checking is not limited to simple report comparison but incorporates hierarchical supervision aimed at ensuring global consistency and analytical robustness. The overall structure of the proposed model is illustrated in Fig. 1.

In this phase of the research, it is only possible to calculate the probability of certain decisions making through the proposed system. In this case the most simplified model of probability theory [15] [16] is applied to illustrate the correct decision made by LLM.

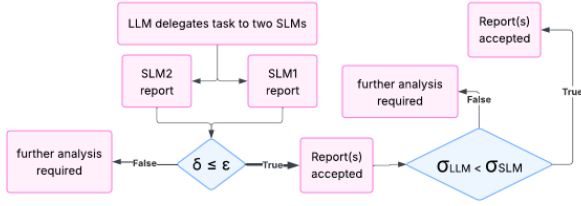


Figure 1. The proposed model of double checking

Applying probability to the model is necessary in order to formally define the relevant events. For analytical tractability, we assume statistical independence between the correctness of the outputs produced by SLM₁ and SLM₂. Under this simplifying assumption, the following events are introduced:

- A – SLM1 produces a correct analysis;
- B – SLM2 produces a correct analysis;
- C – both SLM outputs are in agreement ($\delta \leq \epsilon$);
- D – the two SLM outputs disagree ($\delta > \epsilon$);
- E – the LLM is more reliable than the SLM ($\sigma_{LLM} < \sigma_{SLM}$) and
- F – the SLM is at least as reliable as the LLM ($\sigma_{LLM} \geq \sigma_{SLM}$).

It should be noted that agreement between SLM outputs (event C) does not necessarily imply correctness, but under the independence assumption it increases the likelihood that both analyses are correct. Events C and D are complementary, as are events E and F.

These events are described by their respective probabilities $P(A), P(B), P(C), P(D), P(E)$ and $P(F)$. Under the assumption of statistical independence between the correctness of SLM₁ and SLM₂ outputs, the joint probability that both SLMs produce correct analyses is given by:

$$P(A \cap B) = P(A) * P(B) \quad (4)$$

Event C represents agreement between the two SLM outputs. Although agreement does not necessarily imply correctness, under the independence assumption the probability of correct consensus increases when both analyses are correct..

The probability that the LLM is more reliable than the SLM (event E) is denoted by $P(E)$. A simplified expression for the probability of a positive final decision D^+ , assuming independence between correctness and supervisory reliability, can be approximated as:

$$P(D^+) \approx P(A) * P(B) * P(E) \quad (5)$$

where $P(D^+)$ denotes probability of positive result D^+ .

At this stage of basic idea development, the case D^- (which denotes probability of issues through double checking) was not considered.

If multiple steps in some process of SLMs utilization under supervision of one LLM exist, then the total probability of successful acceptance of reports (in the case of independent events) could be determined as follows:

$$P_t = \prod_{i=1}^n P_i \quad (6)$$

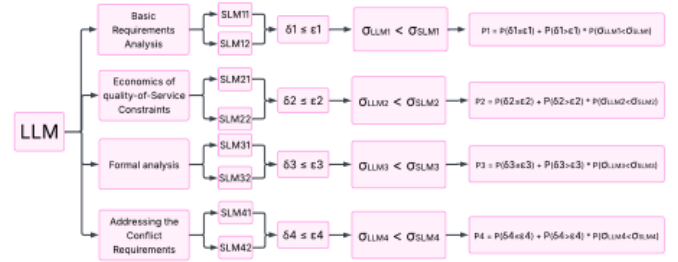
where:

- P_t – total probability of success;
- P_i – probability of successful validation at step i ,
- n – number of subprocesses.

III. RESULTS

In the context of SWEBOOK, software requirements analysis can be decomposed into subprocesses, each incorporating the proposed double-checking mechanism. Although these phases are not strictly independent in practice, for analytical tractability we assume statistical independence between them under LLM supervision (Fig. 2).

Figure 2. The application of model in case of SWEBOOK – Software



Requirements Analysis

As shown in Fig. 2, a final positive report requires successful validation at each subprocess. For each step i , acceptance occurs either through agreement between SLM outputs or through supervisory arbitration. Formally:

$$\{\forall i | (\delta_i \leq \epsilon_i) \vee (\delta_i > \epsilon_i \wedge (\sigma_{LLM} < \sigma_{SLM_i}))\} \quad (7)$$

$$P(D^+) = \prod_{i=1}^4 P_i \quad (8)$$

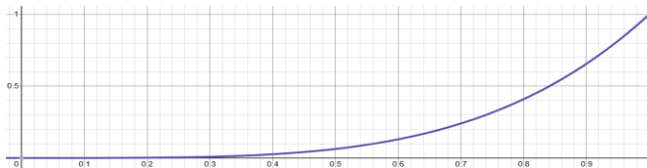
If all subprocesses have equal probability $P_i = P$, then:

$$P(D^+) = P^4 \quad (8)$$

If we accept that probability of every event is $P = 0.5$ it immediately follows that the probability of positive result equals 0.0625 which illustrates the rapid decrease of total

reliability when individual subprocess reliability is moderate. This simple calculation shows that acceptable global reliability can be achieved only if the probability of success at each subprocess is sufficiently high ($P > 0.90$). The presented analysis intentionally simplifies the complex problem of LLM–SLM interaction in software requirements analysis in order to highlight the importance of redundancy and hierarchical supervision for increasing the overall probability of success.

Figure 3. The probability of success when the probability of each event takes different values



IV. DISCUSSION

This research explored the potential of LLM-based MAS to improve efficiency and accuracy in software requirements analysis. Grounded in the SWEBOK view of requirements engineering, the study focused on conceptual challenges that arise when requirements are analyzed manually, including ambiguity, inconsistency, and error propagation. Motivated by recent advances in LLM and SLM, the work investigated how a coordinated, role-based agent model could support key analytical activities and reduce such issues.

The proposed approach introduces task-specialized SLM agents supervised by a higher-level LLM, combined with a double-checking mechanism applied across analysis steps. A simplified probabilistic abstraction was used to illustrate how agreement, disagreement, and supervisory arbitration affect overall analytical reliability, rather than to provide empirical performance estimates. This abstraction highlights the importance of maintaining sufficiently high step-level reliability to avoid cumulative degradation of results.

The presented work represents an early-stage, exploratory contribution. As such, it does not claim empirical validation and is intended to inform future research directions. Further studies should investigate different classes of software requirements, alternative combinations of LLM and SLM agents, and systematic methods for measuring uncertainty, hallucination, and error propagation. Additional research is also required to assess potential limitations of the proposed model and to refine its architectural assumptions.

V. CONCLUSION

This paper reviewed recent research on the application of Large Language Models and multi-agent systems in software requirements analysis, highlighting both their potential and remaining limitations. While LLM-based approaches provide strong support for language-intensive requirements engineering activities, existing studies reveal persistent challenges related to reliability, consistency, hallucination, and error propagation.

To address these issues, the paper outlined a conceptual,

role-based multi-agent vision in which LLM and SLM collaborate through hierarchical supervision and structured cross-checking with uncertainty-aware arbitration. Rather than proposing a fully implemented framework, the contribution of this work lies in synthesizing existing research and formalizing architectural principles and reliability considerations that can guide future development of multi-agent LLM systems for improving the software requirements analysis process.

REFERENCES

- [1] Mahaju, S., Carver, J., C., Bradshaw, G., L., Human error management in requirements engineering: Should we fix the people, the processes, or the environment?, Information and Software Technology, Volume 160, 2023, 107223, ISSN 0950-5849, <https://doi.org/10.1016/j.infsof.2023.107223>.
- [2] <https://www.computer.org/education/bodies-of-knowledge/software-engineering/topics#1> (accessed: 15.01.2026)
- [3] Zadenoori, M. A., Dąbrowski, J., Alhoshan, W., Zhao, L., & Ferrari, A. (2025). Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. arXiv preprint. <https://doi.org/10.48550/arXiv.2509.11446>
- [4] X. Li, S. Zhang, J. Cheng, W. Wu, Y. Peng, C. Gao, L. Chen, and Z. Lin, "A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges," *Vicinaearth*, vol. 1, no. 1, p. 9, 2024.
- [5] Hou, Xinyi, et al. "Large language models for software engineering: A systematic literature review." *ACM Transactions on Software Engineering and Methodology* 33.8 (2024): 1-79.
- [6] He, Junda, Christoph Treude, and David Lo. "LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead." *ACM Transactions on Software Engineering and Methodology* 34.5 (2025): 1-30.
- [7] Tran, K. T., Dao, D., Nguyen, M. D., Pham, Q. V., O'Sullivan, B., & Nguyen, H. D. (2025). Multi-agent collaboration mechanisms: A survey of llms. arXiv preprint arXiv:2501.06322.
- [8] Qian Wang, Tianyu Wang, Zhenheng Tang, Qimbin Li, Nuo Chen, Jingsheng Liang, and Bingsheng He. 2025. Mega Agent: A Large-Scale Autonomous LLM-based Multi-Agent System Without Predefined SOPs. In Findings of the Association for Computational Linguistics: ACL 2025, pages 4998–5036, Vienna, Austria. Association for Computational Linguistics.
- [9] Yang, Y., Tang, C., Song, Q., Xie, R., & Deng, S. (2025). Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems. arXiv preprint arXiv:2504.00587.
- [10] Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., Zhang, C., Jiang, X., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. arXiv preprint arXiv:2402.01680.
- [11] Xu, C., Xu, Y., Wang, S., Liu, Y., Zhu, C., & McAuley, J. (2024, August). Small models are valuable plug-ins for large language models. In Findings of the Association for Computational Linguistics: ACL 2024 (pp. 283-294).
- [12] Schick, T., & Schütze, H. (2021, June). It's not just size that matters: Small language models are also few-shot learners. In Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: Human language technologies (pp. 2339-2352).
- [13] Sami, M.A. et al. (2026). A Multi-agent LLM System for Automated Requirements Analysis: A Study on User Story Generation and Prioritization. In: Taibi, D., Smite, D. (eds) Software Engineering and Advanced Applications. SEAA 2025. Lecture Notes in Computer Science, vol 16082. Springer, Cham. https://doi.org/10.1007/978-3-032-04200-2_12
- [14] Hong, Sirui, et al. "MetaGPT: Meta programming for a multi-agent collaborative framework." *The twelfth international conference on learning representations*. 2023.
- [15] S. S. Varadhan, *Probability Theory*. Providence, RI, USA: American Mathematical Society, 2001.
- [16] W. Feller, *An Introduction to Probability Theory and Its Applications*, vol. 2, 2nd ed. New York, NY, USA: John Wiley & Sons, 1991.